Université de Monastir
Institut Supérieur d'Informatique et de Mathématiques

Cours: Conception par Contrats

Filière: MR1-GL

Chapitre 1: Vérification des Contrats Architecturaux

Réalisé par:
Dr. Sakka Rouis Taoufik

1

I. Introduction à la conception architecturale

- La conception architecturale est une étape clé du développement logiciel
- Elle fait le lien entre l'analyse des besoins et la conception détaillée
- Définition de la structure globale du système à travers l'identification des composants principaux et des interactions entre eux

➔ Composant: un nouveau paradigme de développement «!post-objets!» qui répond à certaines faiblesses de l'approche orientée objet.

2

I. Introduction à la conception architecturale

- La communauté scientifique a développé plusieurs ADLs (Architecture Description Language) permettant la description plus ou moins précise des architecturales à base de composants.
- Les ADL les plus connus: **Wright (Allen, 1997), ACME (Garlan, 2000), Rapide (Luckham, 1995), Unicon (Shaw, 1995), C2 (Taylor, 1996), Darwin (Magee, 1995)** et **AESOP (Garlan, 1994).**

- Généralement, tout ADL doit supporté les 3 concepts de base: **Composant, Connecteur et Configuration**.
- Il peut fournit des concepts supplémentaires: propriétés spécifique, types, contraintes, etc.

I. Introduction à la conception architecturale

1) Composant

- Module logiciel autonome pouvant être installé sur différentes plates-formes.
- Briques configurables pour permettre la construction d'une application par composition
- Une unité de calcul ou de données qui peut être atomique ou composite et qui possède des interfaces (appelés ports) décrivant les points d'interaction du composant avec l'environnement.
- Les modèles de composants hiérarchiques comme Wright, Acme et UML2.0 acceptent le concept de composant composite
- Composants célèbres: Java Beans, Entreprise Java Beans, composants Corba

I. Introduction à la conception architecturale

2) Connecteur

- modélise les interactions entre les composants et les règles qui régissent ces interactions.

-définit un ensemble d'interfaces, appelés **rôles** qui permettent d'identifier les participants à l'interaction.

Par exemple, un connecteur «Pipe» possède un rôle d'écriture et un rôle de lecture.

I. Introduction à la conception architecturale

3) Configuration

- décrit la structure complète d'un système sous forme d'un graphe connexe regroupant des composants et des connecteurs.

- Le graphe est obtenu en associant les ports des composants avec les rôles des connecteurs adéquats, en vue de construire l'application.

Par exemple, les ports des composants de type "Filtre" sont associés aux rôles des connecteurs de type "Pipe" à travers lesquels ils lisent et écrivent des flux de données.

I. Introduction à la conception architecturale

4) Concepts supplémentaires

Propriétés: La capacité de spécifier une propriété d'un élément architectural. Par exemple, L'ADL Acme supporte le concept Property qui permet la spécification d'une propriété sous la forme d'un triplet (nom, type et valeur).

Types: Le typage permet l'encapsulation des fonctionnalités et des éléments internes en vu d'être réutilisables.

Contraintes: expriment des restrictions placées sur les composants, les connecteurs et les configurations. Une contrainte sur un composant peut par exemple servir à borner les valeurs de ses propriétés.

I. Introduction à la conception architecturale

Avantages:

- **Spécialisation :** L'équipe peut-être divisée en sous-groupes, chacun se spécialisant dans le développement d'un composant.
- **Sous-traitance :** Le développement d'un composant peut-être externalisé, à condition d'en avoir bien réalisé les spécifications au préalable
- **Facilité de mise à jour :** La modification d'un composant ne nécessite pas la recompilation du projet complet

I. Introduction à la conception architecturale

Avantages:

- **Choix des langages de développement :** Il est possible, dans la plupart des cas, de développer les différents composants du logiciel dans des langages de programmation différents.

- **Productivité :** La réutilisabilité d'un composant permet un gain de productivité non négligeable car elle diminue le temps de développement, d'autant plus que le composant est réutilisé souvent

I. Introduction à la conception architecturale

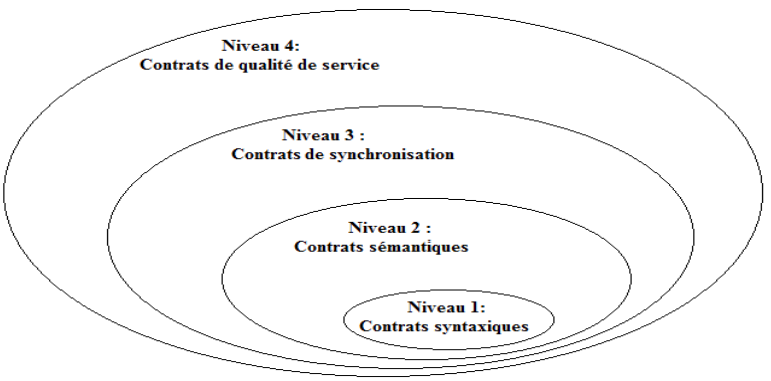
Inconvenants:

- Factoriser un logiciel en composants nécessite un travail important dans :
  - **phase d'analyse**
  - **phase de conception architecture**.

- Un assemblage sain de composants exige **la vérification des propriétés (contrats)** liées à la cohérence des interfaces interconnectées.

II. Classification des contrats dans le cadre O. C.

➔ A. Beugnard et al. (1999) ont établi une classification des contrats des composants selon quatre niveaux:



II. Classification des contrats dans le cadre O. C.

1) Les contrats syntaxiques:

permettent de vérifier la conformité entre les signatures des méthodes des interfaces. La signature d'une méthode peut comporter les éléments suivants :

- nature de méthode : Opération de construction, consultation et modification.
- paramètres formels : Pour chaque paramètre, trois informations à prendre en considération à savoir son type, sa position et sa nature logique (in, out et in/out).

II. Classification des contrats dans le cadre O. C.

2) Les contrats sémantiques:

les propriétés qui peuvent être spécifiées avec des pré-conditions, des post-conditions et des invariants.

Ces propriétés sont liées à une méthode, qui est vue comme une unité élémentaire.

Les travaux sur la conception par contrats (Meyer, 1992) se placent à ce niveau.

Une telle propriété sémantique peut être décrite en utilisant un langage de contraintes de type OCL.

II. Classification des contrats dans le cadre O. C.

3) Les contrats comportementaux

Il s'agit ici des propriétés concernant les interactions (la synchronisation, la concurrence, la coopération) des éléments architecturaux (composants, ports, connecteurs etc.).

Le comportement global ou partiel des éléments architecturaux peut être décrit par:

- des PSMs (Protocol State Machine cas d'UML2.0)
- des algèbres de processus (CSP pour l'ADL Wright).
- des taches concurrentes (rendez-vous de tasks Ada)

La vérification des contrats comportementaux (de synchronisation) est souvent confiée à des model-checkers.

II. Classification des contrats dans le cadre O. C.

4) Les contrats qualitatifs

couvrent les propriétés non-fonctionnelle telles que:

la qualité de service:

- fiabilité,
- disponibilité,
- sécurité,
- etc.

la gestion des ressources,

le temps de réponse.

III. Introduction à l'ADL Acme/Armani

Concepts structuraux:

- Component, Connector, Port, Role, System, Family

Possibilités de typage :

- Types prédéfinis (int, float, boolean et string)
- Constructeurs de types simples (enum)
- Constructeurs de types composés (record, set et sequence)

Formalisation des propriétés spécifiques aux AL telles que les propriétés structurelles et les propriétés qualitatives

III. Introduction à l'ADL Acme/Armani

Langage de prédicats Armani

- Fonctions prédéfinies appropriées au domaine d'architecture logicielle : Fonctions de conception, Fonctions d'analyse, …
- Propriétés architecturales sous forme d'**invariants** ou d'**heuristiques** attachées aux différents éléments architecturaux.

Environnement AcmeStudio

- Environnement de développement doté d'un outil d'analyse et de validation des contraintes Armani

Acme/Armani favorise la formalisation et la vérification des propriétés structurelles et qualitatives des AL.

Ch 2: Conception par contrats dans le cadre O. C.

III. Introduction à l'ADL Acme/Armani

Fonctions prédéfinies de Armani:

Les fonctions de type : ces fonctions concernent les types d'éléments. Par exemple, la fonction **"DeclaresType (E :Element, T :Type) :boolean"** permet de vérifier si l'élément **E** est de type **T** ou non.

Les fonctions de graphe : ces fonctions concernent la connectivité d'éléments. Par exemple, la fonction **"Attached (E1 : Element, E2 :Element) :boolean"** permet de vérifier si l'élément **E1** est directement attaché à l'élément **E2** ou non.

Les fonctions de propriété : ces fonctions concernent les propriétés. Par exemple, la fonction **"HasValue (P :Property) :boolean"** permet de vérifier si la propriété **P** possède une valeur ou non.

Les fonctions d'ensemble : ces fonctions regroupent les fonctions appliquées sur les ensembles telles que **: union, intersection, contains, size, sum, select, collect,** etc.

III. Introduction à l'ADL Acme/Armani

Opérateurs de Armani

les opérateurs de comparaison (==, >, <,…),
les opérateurs arithmétique (+, -, /, mod, ...),
les opérateurs logique (and, or, →, …)
l'opérateur de qualification (•).

En plus, il supporte d'autres opérateurs spécifiques au domaine d'architecture logicielle telles que:
Components, Connectors, Ports, AttachedPorts, AttachedRoles, etc.

Chaque opérateur spécifique est appliqué sur une catégorie d'éléments.

Par exemple, l'opérateur **"Ports"** est appliqué sur un élément de catégorie "Component" et permet de retourner l'ensemble de tous les ports de ce composant.

Ch 2: Conception par contrats dans le cadre O. C.

III. Introduction à l'ADL Acme/Armani

Opérateurs de Armani

- l'operateur **Components** appliqué sur un élément de catégorie **système (ou famille)** permet de retourner l'ensemble de tous les composants de ce système.
- l'operateur **Connectors** appliqué sur un élément de catégorie **système (ou famille)** permet de retourner l'ensemble de ses connecteurs.
- l'operateur **Ports** appliqué sur un élément de catégorie **component** permet de retourner l'ensemblede ses ports.
- l'operateur **Roles** appliqué sur un élément de catégorie **conector** permet de retourner l'ensemble de ses roles.
- l'operateur **AttachedPorts** appliqué sur un élément de catégorie **role** permet de retourner l'ensemble des ports attachés a ce role.

- l'operateur **AttachedRoles** appliqué sur un élément de catégorie **Port** permet de retourner l'ensemble des ports attachés a ce port.

III. Introduction à l'ADL Acme/Armani

Quantificateurs de Armani

Armani supporte deux quantificateurs (**Forall** et **Exists**) qui permettent la quantification d'un prédicat.

Exemples d'expressions quantifiées en Armani

Forall comp : CompType **in** sys**.**Components **|** comp.secure **= true;**

Exists conn : Connector **in** sys**.**Connectors **| declareType** (conn, EventSystemType)**;**

Ch 2: Conception par contrats dans le cadre O. C.

III. Introduction à l'ADL Acme/Armani

Fonctions de conception de Armani

Armani offre la possibilité à l'architecte de définir des fonctions appropriées à son application. Ces fonctions sont appelées des fonctions de conception. Une fonction de conception est une simple fonction qui peut être utilisée dans la définition des contraintes architecturales.

Armani offre deux constructions permettant la définition des fonctions de conception. Le premier (**analysis**), permet la définition de fonctions de conception toute en utilisant les concepts Acme/Armani. Le second (**external analysis**), permet d'ouvrir Acme/Armani sur d'autres langages souvent impératifs comme Java.

III. Introduction à l'ADL Acme/Armani

L'outil AcmeStudio

Acme est supporté par un environnement de développement appelé AcmeStudio.

Lien de l'outil:

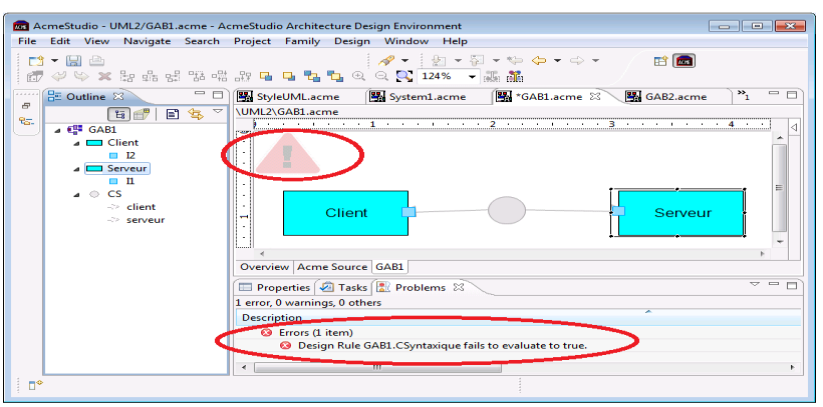
http://acme.able.cs.cmu.edu/AcmeStudio/

Ch 2: Conception par contrats dans le cadre O. C.

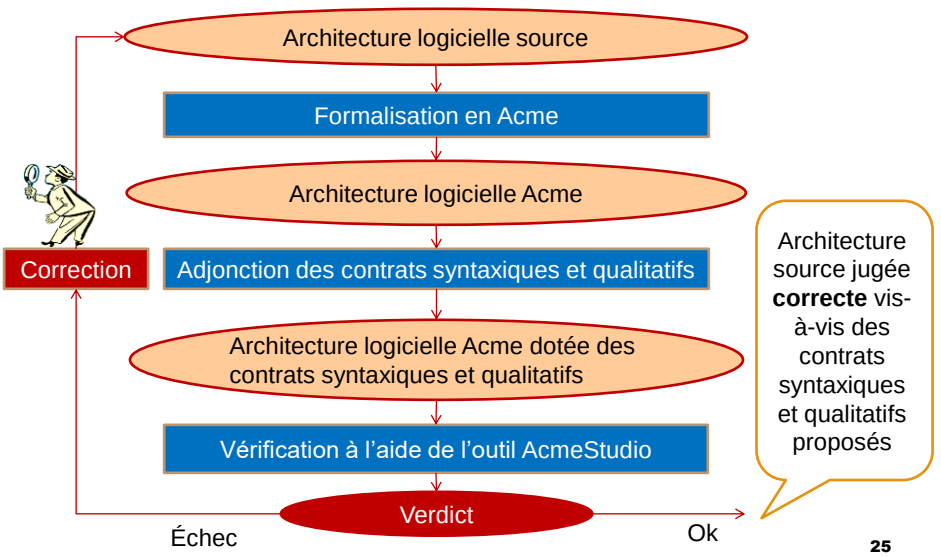
III. Introduction à l'ADL Acme/Armani

L'outil AcmeStudio

AcmeStudio implémente un outil d'analyse et de validation des contraintes Armani : évaluateur des contraintes Armani.



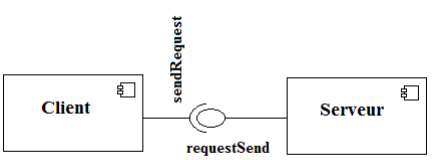
IV. Utilisation de l'ADL Acme/Armani comme machine de vérification des contrats syntaxiques et qualitatifs



IV. Utilisation de l'ADL Acme/Armani comme machine de vérification des contrats syntaxiques et qualitatifs

- Exemple 1:

Une simple description architecturale d'un système client-serveur en UML2.0



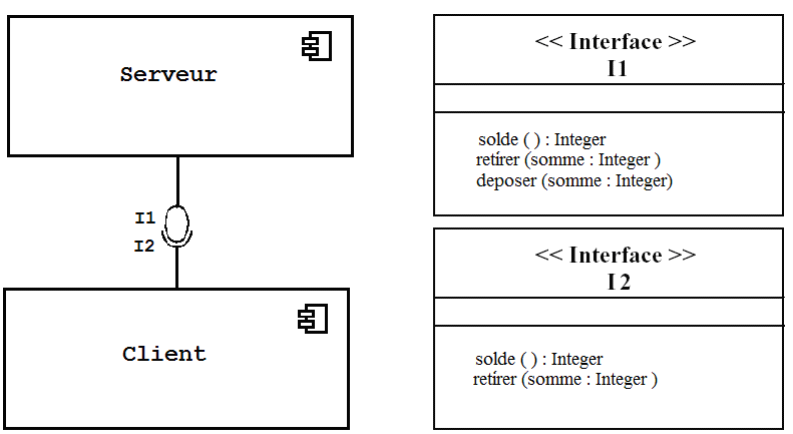
IV. Utilisation de l'ADL Acme/Armani comme machine de vérification des contrats syntaxiques et qualitatifs

- Exemple 1: le système client-serveur en Acme

```
System simple_cs = {
        Component Client = {
                Port sendRequest ;
        }
        Component Server = {
                Port requestSend ;
        }
        Connector RPC = {
                Role caller ;
                Role callee ;
        }
        Attachment Client.sendRequest to RPC.caller ;
        Attachment Server.requestSend to RPC.callee ;
}
```

Ch 2: Conception par contrats dans le cadre O. C.

IV. Utilisation de l'ADL Acme/Armani comme machine de vérification des contrats syntaxiques et qualitatifs



IV. Utilisation de l'ADL Acme/Armani comme machine de vérification des contrats syntaxiques et qualitatifs

- Exemple 2: Le système GAB en Acme

```
System GAB = {
        Component Client = {
                Port I2={
                        Property solde: String;
                        Property retirer: String; }
        }

        Component Server = {
                Port I1={
                        Property solde: String;
                        Property retirer: String;
                        Property deposer: String; }
        }

        Connector RPC = {
                Role R1 ;
                Role R2 ;
        }
        Attachment Client.I2 to RPC.R1;
        Attachment Server.I1  to RPC.R2;
}
```

Ch 2: Conception par contrats dans le cadre O. C.

IV. Utilisation de l'ADL Acme/Armani comme machine de vérification des contrats syntaxiques et qualitatifs

Style Architecturale:

```
Family SCUML = {

//type des interfaces UML2.0
        Port Type InterfaceUML = {……————};

//type des composants UML2.0
Component type ComponentUML = {————————————};

//type des connecteurs  UML2.0
        Connector type ConnectorUML = {————————————};

//ensemble de fonctions de conception
}
```

IV. Utilisation de l'ADL Acme/Armani comme machine de vérification des contrats syntaxiques et qualitatifs

Style Architecturale:

L'intérêt principal de l'architecture logicielle est d'utiliser des **patrons de conception** récurrents : les styles architecturaux.

Un style architectural peut être considéré comme un type d'un ensemble d'architectures qui partagent certaines propriétés entre eux.

Il permet de décrire un vocabulaire commun en définissant un ensemble de types de concepts structurels (ports, composant, connecteur…) et un ensemble de propriétés et de contraintes partagées par toutes les configurations appartenant à ce style.

IV. Utilisation de l'ADL Acme/Armani comme machine de vérification des contrats syntaxiques et qualitatifs

Style Architecturale:

```
property type TypeInterface = enum {provided, required};

  property type Services = set {string};

  port type InterfaceUML = {

      property  typeInterface: TypeInterface;

      property services: Services;
  }
```

IV. Utilisation de l'ADL Acme/Armani comme machine de vérification des contrats syntaxiques et qualitatifs

Style Architecturale:

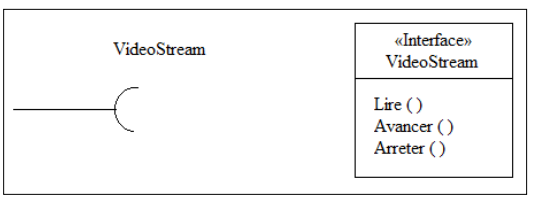
//Utitlisation au niveau systeme

Port VideoStream : InterfaceUML= new InterfaceUML **extended with** {

Property typeInterface = required;

Property services = {"Lire", "Avancer", "Arreter"} ;

};



IV. Utilisation de l'ADL Acme/Armani comme machine de vérification des contrats syntaxiques et qualitatifs

Style Architecturale:

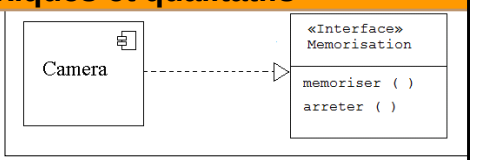
component type ComponentUML ={

rule RCcomposant1= **invariant**
forall p:**port in** self.**ports**| declaresType (p, InterfaceUML);

}

IV. Utilisation de l'ADL Acme/Armani comme machine de vérification des contrats syntaxiques et qualitatifs

Style Architecturale:



// utilisation au niveau système

Component Camera : ComponentUML = **new** ComponentUML **extended with** {

Port Memorisation : InterfaceUML= new InterfaceUML **extended with** {

Property typeInterface = provided ;

Property services = {"memoriser", "arreter"} ;
};
}

IV. Utilisation de l'ADL Acme/Armani comme machine de vérification des contrats syntaxiques et qualitatifs

Style Architecturale:

//utilisation du style
import families/SCUML.acme;
System ClientServeur : SCUML = **new** SCUML **extended with** {
Component Client : ComponentUML = **new** ComponentUML **extended with** {

Port I1 : InterfaceUML= new InterfaceUML **extended with** {

Property typeInterface = required ;

Property services = { .....} ;
};
}

IV. Utilisation de l'ADL Acme/Armani comme machine de vérification des contrats syntaxiques et qualitatifs

Style Architecturale:

Étendre le style SCUML par les contraintes suivantes:
RCcomposant1 : tous les ports de ce composant doivent être de type «InterfaceUML».

- RCconnecteur1 : tout connecteur UML doit proposer deux rôles (client et serveur)
- RCconnecteur2 : le rôle client doit être attaché à un seul port et qui doit être obligatoirement de type interface requise.
- RCconnecteur3 : le rôle serveur doit être attaché à un seul port et qui doit être obligatoirement de type interface offerte.

CSyntaxique : un connecteur d'assemblage entre une interface requise et une autre offerte est dit sain si et seulement si l'ensemble des services requis (venant de l'interface requise) est inclus dans l'ensemble des services offerts (venant de l'interface offerte). De plus, toute interface requise est forcément attachée à une interface offerte.